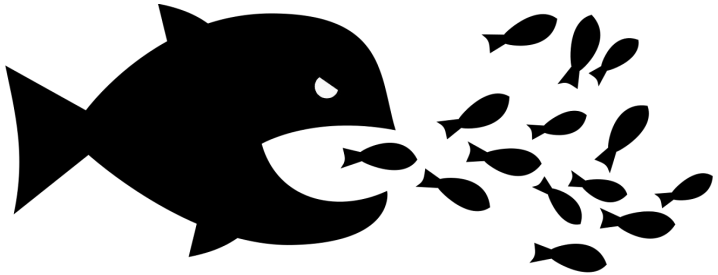
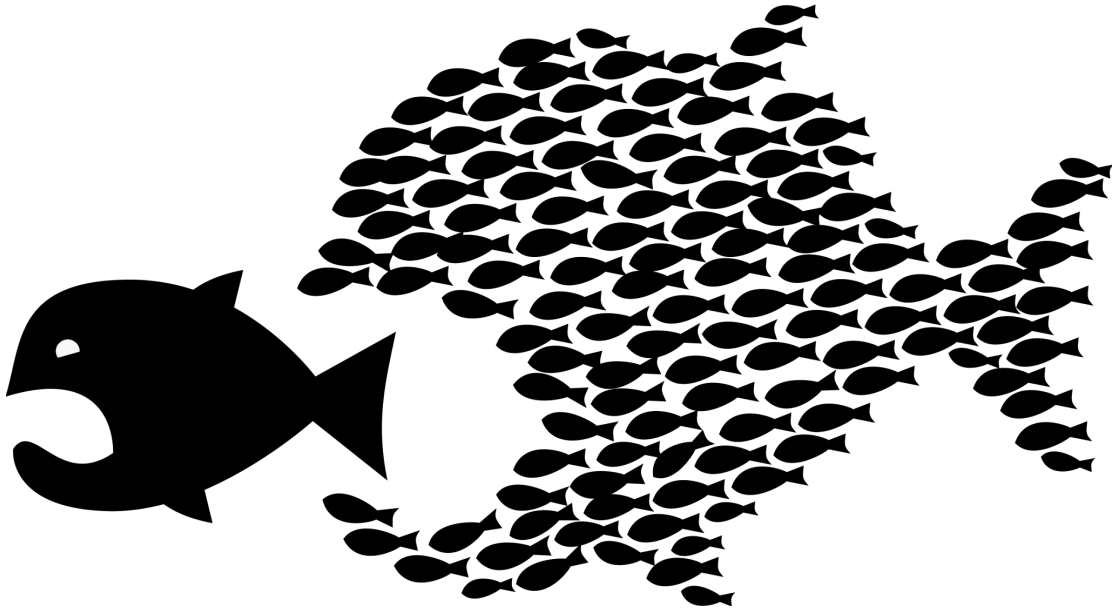
Introducción a la Ciencia de Datos

Maestría en Ciencias de la Computación

Dr. Irvin Hussein López Nava

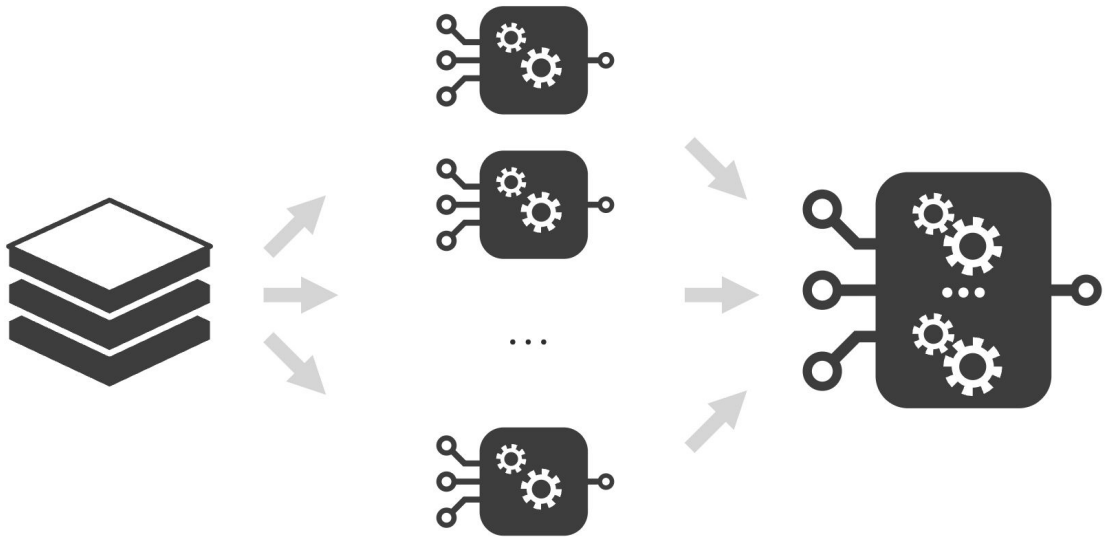
Laboratorio de Ciencia de Datos y Aprendizaje Automático





05

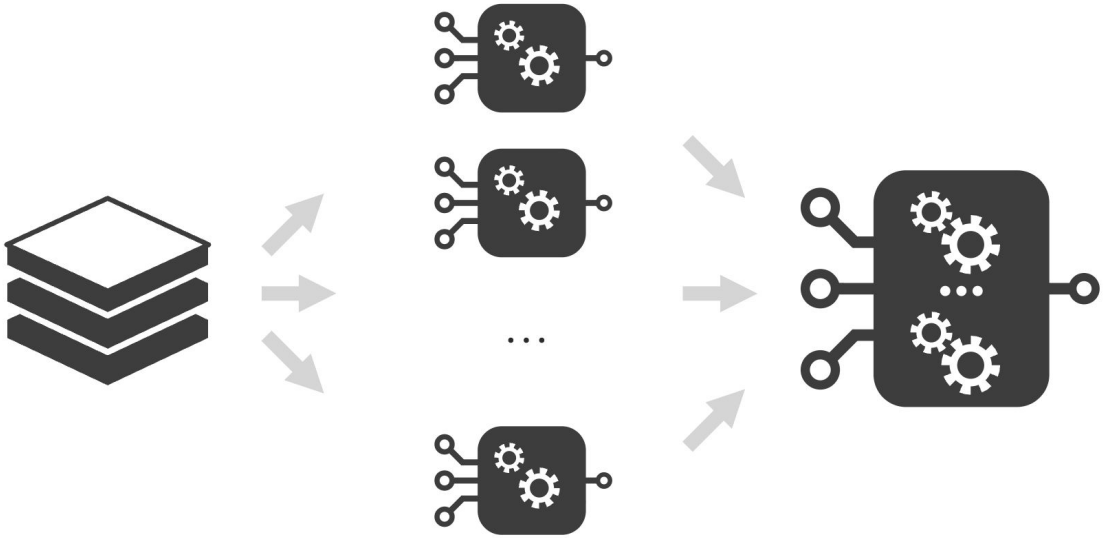
Meta-aprendizaje



dataset inicial

L modelos distintos
entrenados con el mismo
dataset completo

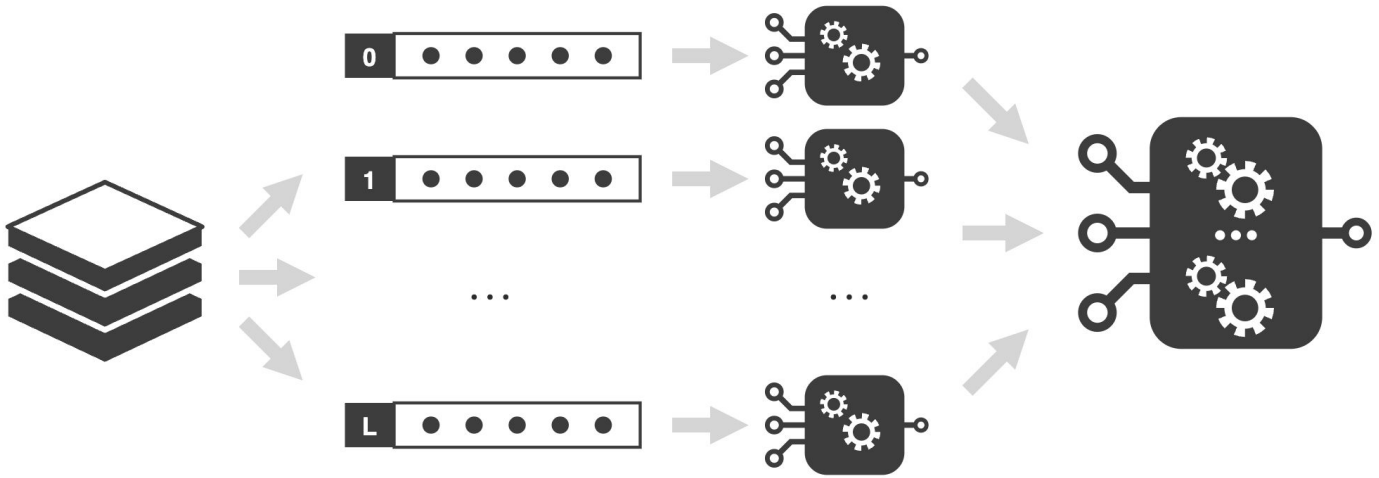
modelo de ensamble
(voto mayoritario o promedio de las
predicciones; **sin entrenamiento**)



dataset inicial

L modelos débiles
(pueden ser heterogéneos)

metamodelo
(**entrenado** para predecir a partir
de las predicciones de los modelos débiles)

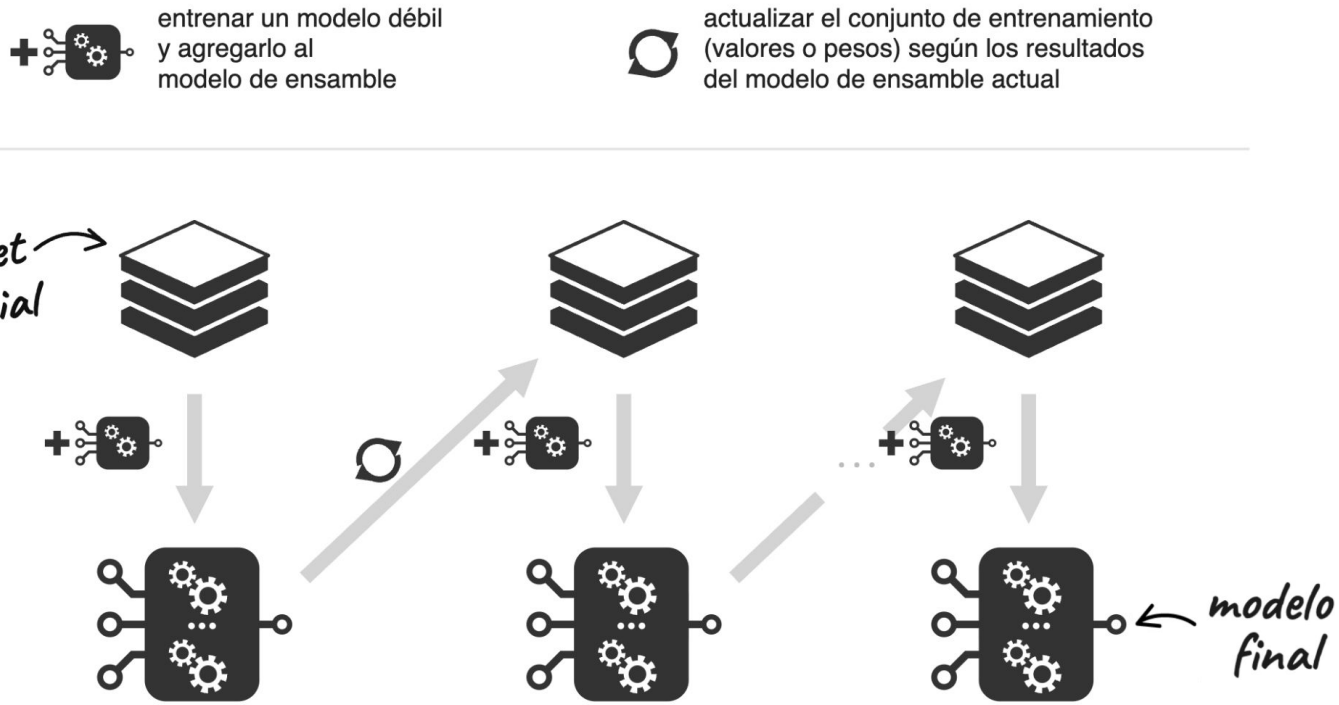


dataset inicial

L muestras bootstrap

modelos débiles entrenados en cada muestra bootstrap

modelo de ensamble (una especie de promedio de los modelos débiles)


entrenar un modelo débil
y agregarlo al
modelo de ensamble
actualizar el conjunto de entrenamiento
(valores o pesos) según los resultados
del modelo de ensamble actual
dataset
inicial
modelo
final


The latest LLM
XGBoost

5.2 Boosting

Centrarse en el boost

- En los **métodos secuenciales**, los diferentes modelos débiles combinados no se ajustan de forma independiente unos de otros.
- La idea es ajustar los modelos de forma **iterativa**, de modo que el entrenamiento del modelo en un paso determinado dependa de los modelos ajustados en los pasos anteriores.
- El *boosting* produce un modelo de ensamble que, en general, está menos **sesgado** que los aprendices débiles que lo componen.

Boosting vs Bagging

- **Ambos métodos** funcionan con el mismo espíritu:
  - Construyen un conjunto de modelos agregados para obtener un aprendiz fuerte que obtenga mejores resultados.
- Sin embargo, a diferencia del **bagging**, cuyo principal objetivo es reducir la varianza, el **boosting** consiste en ajustar secuencialmente múltiples aprendices débiles de forma adaptativa:
  - Cada modelo se ajusta dando más importancia a las observaciones del conjunto de datos que no fueron bien “tratadas” por los modelos anteriores en la secuencia.
- Al final del proceso se obtiene un aprendiz fuerte con un sesgo menor.

Modelos base para Boosting

- Al centrarse en la reducción del **sesgo**, los modelos base que se consideran para *boosting* son aquellos con baja varianza pero alto sesgo,
  - e.g., árboles de decisión poco profundos.
- Otra razón para usar este tipo de modelos en *boosting* es que, en general, su ajuste es menos costoso desde el punto de vista computacional.
  - Dado que los cálculos no pueden realizarse en paralelo (a diferencia de *bagging*), podría resultar demasiado costoso ajustar secuencialmente varios modelos complejos.
- Las preguntas: ¿cómo se ajustarán secuencialmente los modelos? y ¿cómo se agregarán tales modelos?

Adaptive Boosting

- En el *boosting* adaptativo, también llamado **adaboost**, un modelo de ensamble se define como una suma ponderada de *L* aprendices débiles:

$$s_L(x) = \sum_{l=1}^{L} c_l w_l(x)$$

  donde *c* son los coeficientes y *w* son los modelos base con *x* como entrada.
- Encontrar el mejor modelo de ensamble es un problema de optimización.
- En lugar de tratar de encontrar todos los coeficientes y aprendices débiles que den el mejor modelo aditivo global, se utiliza un proceso de optimización iterativo, aunque pueda conducir a una solución subóptima.

Adaptive boosting

- Se añaden aprendices débiles uno a uno, buscando en cada iteración el mejor par posible (coeficiente – aprendiz débil) para añadir al modelo de ensamble actual.

$$s_l(x) = s_{l-1}(x) + c_l w_l(x)$$

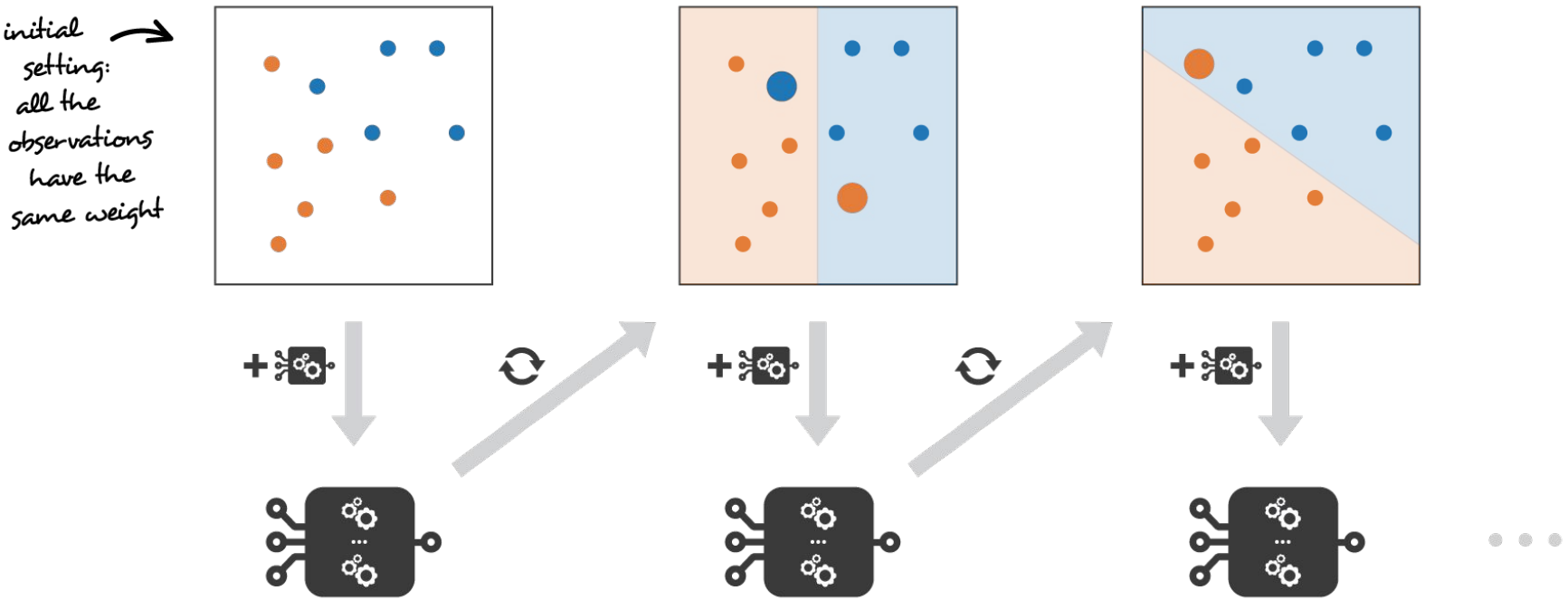
donde c_l y w_l se eligen de tal forma que s_l es el modelo que mejor se ajusta a los datos de entrenamiento y es la mejor posible mejora sobre s_{l-1}.

$$(c_l, w_l(x)) = arg\ \underset{c_l, w_l(x)}{min} \sum_{n=1}^{N} e(y_n, s_{l-1}(x_n) + cw(x_n))$$

donde *e* es la función de pérdida/error.

Adaptive boosting

- En lugar de optimizar globalmente sobre todos los modelos *L* de la suma, se aproxima el óptimo localmente añadiendo uno a uno los aprendices débiles a un modelo fuerte.
- Cuando se considera una clasificación binaria, el algoritmo **adaboost** puede reescribirse en un proceso como el siguiente:
    1. Actualizar los pesos de las observaciones en el conjunto de datos y entrenar un nuevo aprendiz débil con especial atención a las observaciones mal clasificadas por el modelo de ensamble actual.
    2. Añadir el aprendiz débil a la suma ponderada de acuerdo a un coeficiente de actualización que exprese el rendimiento de este modelo débil: cuanto mejor sea el rendimiento de un aprendiz débil, más contribuirá al aprendiz fuerte.


train a weak model and aggregate it to the ensemble model
update the weights of observations misclassified by the current ensemble model
current ensemble model predicts “orange” class
current ensemble model predicts “blue” class
initial setting: all the observations have the same weight

Adaboost algorithm

function ADABOOST($examples, L, K$) **returns** a weighted-majority hypothesis
  inputs: $examples$, set of N labeled examples $(x_1, y_1), \ldots, (x_N, y_N)$
    L, a learning algorithm
    K, the number of hypotheses in the ensemble
  local variables: **w**, a vector of N example weights, initially $1/N$
    h, a vector of K hypotheses
    z, a vector of K hypothesis weights

  for $k = 1$ **to** K **do**
    $\mathbf{h}[k] \leftarrow L(examples, \mathbf{w})$
    $error \leftarrow 0$
    for $j = 1$ **to** N **do**
      if $\mathbf{h}[k](x_j) \neq y_j$ **then** $error \leftarrow error + \mathbf{w}[j]$
    for $j = 1$ **to** N **do**
      if $\mathbf{h}[k](x_j) = y_j$ **then** $\mathbf{w}[j] \leftarrow \mathbf{w}[j] \cdot error/(1 - error)$
    $\mathbf{w} \leftarrow$ NORMALIZE(**w**)
    $\mathbf{z}[k] \leftarrow \log(1 - error)/error$
  return WEIGHTED-MAJORITY(**h**, **z**)

El mismo algoritmo, escrito con α

Para $t = 1, \ldots, T$:

1. Entrenar h_t con los pesos w.
2. Error ponderado ε_t: la suma de los pesos de los mal clasificados.
3. Peso del modelo: $\alpha_t = \frac{1}{2}\ln\frac{1-\varepsilon_t}{\varepsilon_t}$
4. Repesar y normalizar: $w_i \leftarrow \frac{w_i\, e^{-\alpha_t\, y_i\, h_t(x_i)}}{Z_t}$

Clasificador final: $H(x) = \operatorname{signo}\left(\sum_{t=1}^{T} \alpha_t\, h_t(x)\right)$

	Peso del modelo	**Repesar, antes de normalizar**
Diapositiva 17	$z = \ln((1-\varepsilon)/\varepsilon)$	bien × $\varepsilon/(1-\varepsilon)$
Con α	$\alpha = \frac{1}{2}\ln((1-\varepsilon)/\varepsilon)$	bien × $e^{-\alpha}$, mal × e^{α}
scikit-learn	$\ln((1-\varepsilon)/\varepsilon) + \ln(K-1)$	mal × $(K-1)(1-\varepsilon)/\varepsilon$

Con dos clases ($K = 2$) es el mismo algoritmo: z y el peso de scikit-learn valen 2α, y multiplicar por 2 todos los pesos del voto no cambia el signo de la suma. Las tres reglas dan los mismos pesos al normalizar: en las tres, el peso de un mal clasificado crece $(1-\varepsilon)/\varepsilon = e^{2\alpha}$ veces respecto al de uno bien clasificado.

AdaBoost a mano: la primera ronda

- Los 14 días empiezan con peso 1/14 = 0.071.
  Outlook y Humidity empatan con 4 errores; tomamos **Outlook**:
  sunny → No (3 de 5), overcast → Sí (4 de 4), rain → Sí (3 de 5).
  Falla los días 9 y 11 (sunny, sí se jugó) y 6 y 14 (rain, no).
- Mal clasificados × e^{α} = 1.581; bien clasificados × $e^{-\alpha}$ = 0.632;
  Z_1 normaliza:

$$\varepsilon_1 = \frac{4}{14} = 0.286 \Rightarrow$$

$$\alpha_1 = \frac{1}{2}\ln\frac{0.714}{0.286} = \frac{1}{2}\ln 2.5 = 0.458$$

$$\frac{1}{14}e^{0.458} = 0.1129 \qquad \frac{1}{14}e^{-0.458} = 0.0452$$

$$Z_1 = 4(0.1129) + 10(0.0452) = 0.9035$$

día

	1	2	3	4	5	**6**	7	8	**9**	10	**11**	12	13	**14**
antes	.071	.071	.071	.071	.071	.071	.071	.071	.071	.071	.071	.071	.071	.071
después	.050	.050	.050	.050	.050	**.125**	.050	.050	**.125**	.050	**.125**	.050	.050	**.125**

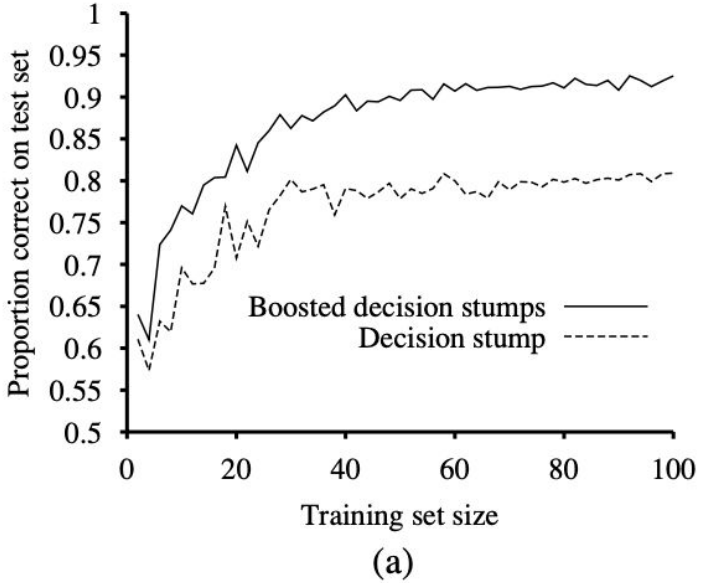
Normalizados, los cuatro días mal clasificados pesan 0.125 cada uno, 0.5 en total.
La ronda 1 queda con error ½, como el azar, y la ronda 2 tiene que buscar otro.

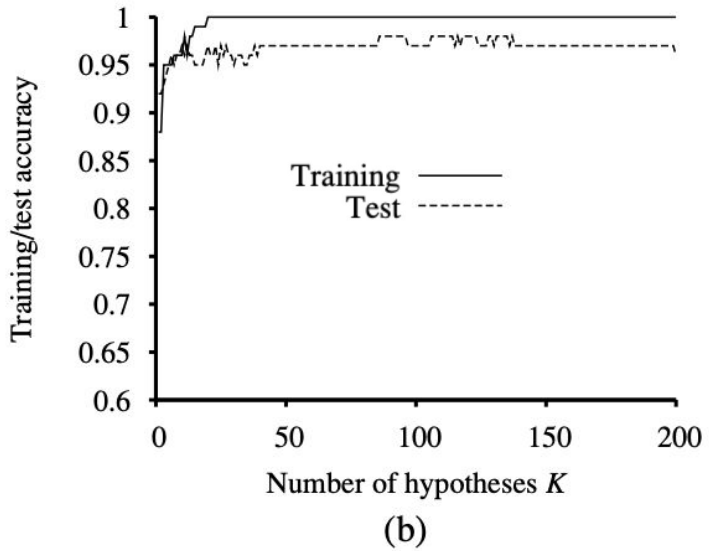
AdaBoost a mano: cinco rondas

Ronda	Tocón (árbol de un nivel)	ε	α	Mal clasificados	Ensamble, entren.	Ensamble, prueba
1	Outlook: **sunny No**, overcast Sí, rain Sí	0.286	0.458	6, 9, 11, 14	10/14	10/14
2	Humidity: **high No**, normal Sí	0.275	0.485	3, 4, 6, 12	10/14	7/14
3	Outlook: **sunny Sí**, overcast Sí, rain No	0.263	0.514	1, 2, 4, 5, 8, 10	12/14	10/14
4	Wind: weak Sí, **strong No**	0.275	0.486	1, 7, 8, 11, 12	11/14	9/14
5	Outlook: **sunny No**, overcast Sí, rain Sí	0.294	0.439	6, 9, 11, 14	12/14	12/14

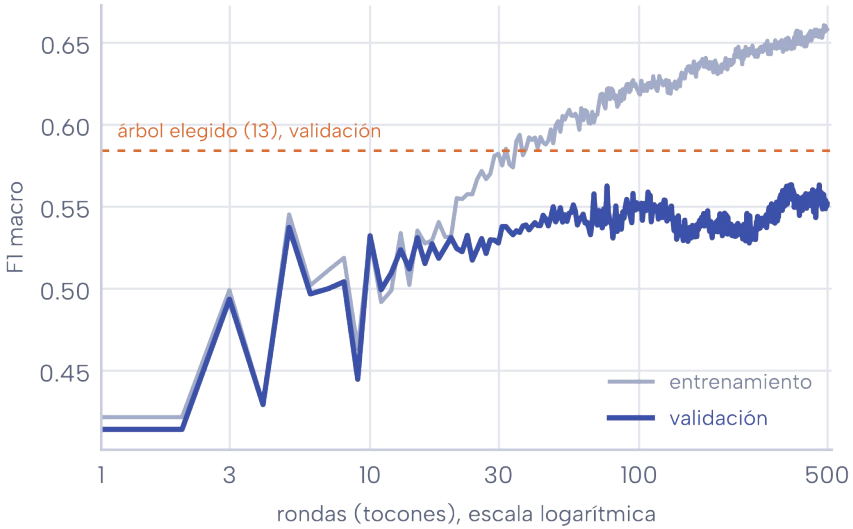
- La ronda 3 vuelve a preguntar Outlook, con sunny y rain al revés: es el tocón que mejor acierta los días que ahora pesan más.
- La consulta de la sesión 14, {*sunny, cool, high, strong*}, suma −0.458 − 0.485 + 0.514 − 0.486 − 0.439 = −1.35 < 0: **No**, igual que con Naive Bayes.
- Con cinco tocones, el ensamble acierta 12 de 14 días de entrenamiento y 12 de 14 de prueba (falla el 23 y el 27). En prueba sube y baja de una ronda a otra: 14 días son muy pocos para leer la curva.

Ejemplo de ejecución Adaboost





AdaBoost en el vino



Tocones, 500 rondas: F1 macro, promedio de los 5 pliegues.

Árbol base	Rondas	Entren.	Validación	Prueba
tocón	500	0.658	0.551 ± 0.032	0.550
profundidad 2	500	0.837	0.572 ± 0.033	0.578
profundidad 3	500	0.918	0.567 ± 0.021	0.571
profundidad 6	500	1.000	0.573 ± 0.035	0.637
sin límite	**1**	1.000	0.558 ± 0.031	0.490

- Con tocones, la validación sube de 0.414 a 0.551 en 500 rondas: boosting baja el sesgo del tocón, pero no alcanza al árbol elegido de la sesión 13 (0.584).
- Con profundidad 2 a 6, el entrenamiento sube hasta 1 y la validación apenas se mueve (0.567 a 0.573).
- Un árbol sin límite no se equivoca en entrenamiento: con $\varepsilon = 0$, AdaBoost se detiene con un solo árbol.

Varias clases: SAMME

- Con K clases, adivinar acierta $1/K$: un modelo débil solo necesita $\varepsilon < 1 - 1/K$. Con 3 clases, $\varepsilon < \tfrac{2}{3}$.
- SAMME suma $\ln(K - 1)$ al peso de cada modelo y repesa solo a los mal clasificados:

$$\alpha_t = \ln \frac{1 - \varepsilon_t}{\varepsilon_t} + \ln(K - 1)$$

- En el vino, el tocón de la ronda 1 se equivoca en 41 % del peso: $\alpha = \ln(0.590/0.410) + \ln 2 = 1.058$. Desde la ronda 2, su error ponderado queda entre 0.49 y 0.66: un AdaBoost de dos clases se detendría en ½; SAMME sigue mientras $\varepsilon < \tfrac{2}{3}$.
- En scikit-learn, `AdaBoostClassifier()` usa tocones por omisión y solo implementa SAMME: SAMME.R se quitó en la versión 1.6.

Más sobre Adaboost

- *Adaboost* actualiza los pesos de las observaciones en cada iteración.
- Las ponderaciones de las observaciones bien clasificadas disminuyen en relación con las ponderaciones de las observaciones mal clasificadas.
- Los modelos que obtienen mejores resultados tienen ponderaciones más altas en el modelo conjunto final.
- Puede identificar valores atípicos, ya que se centra en ejemplos difíciles de clasificar.
- Demasiados valores atípicos pueden degradar el rendimiento de la clasificación y aumentar drásticamente el tiempo de convergencia.

Gradient boosting

- En *boosting* por gradiente, el modelo de ensamble a construir es también una suma ponderada de aprendices débiles.

$$s_L(x) = \sum_{l=1}^{L} c_l w_l(x)$$

- Como en el caso de adaboost, encontrar el modelo óptimo de esta forma es demasiado difícil y se requiere un enfoque iterativo.
- La principal diferencia con el *boosting* adaptativo es la definición del proceso de optimización secuencial.

Gradient boosting

- Gradient boosting convierte el problema en uno de descenso de gradiente: en cada iteración se ajusta un aprendiz débil al gradiente opuesto del error de ajuste actual con respecto al modelo de ensamble actual.
- El proceso de descenso de gradiente sobre el modelo de ensamble puede escribirse:

$$s_l(x) = s_{l-1}(x) - c_l \nabla_{s_{l-1}} E(s_{l-1})(x)$$

donde *E* es el error de ajuste del modelo dado y

$$-\nabla_{s_{l-1}} E(s_{l-1})(x)$$

es el gradiente opuesto del error de ajuste en el paso *l–1*.

Gradient boosting

- Lo opuesto al gradiente es una función para las observaciones del conjunto de datos de entrenamiento: estas evaluaciones se denominan **pseudo-residuales** adjuntas a cada observación.
- Se ajusta un aprendiz débil a los pseudo-residuos calculados para cada observación.
- Por último, el coeficiente *c* se calcula siguiendo un proceso de optimización unidimensional (búsqueda lineal para obtener el mejor tamaño de paso *c*).

Gradient boosting

- Al inicio del algoritmo, el modelo es la constante F_0 que minimiza la pérdida: la media de y con error cuadrático; el logaritmo de los momios con log-loss.
- Los pseudo-residuos iniciales son $y - F_0$ (error cuadrático) o $y - p_0$ (log-loss): son las observaciones solo si $F_0 = 0$ y la pérdida es cuadrática.
- A continuación, se repite L veces (modelos de la secuencia):
  - Ajustar el mejor aprendiz débil posible a los pseudo-residuos.
  - Calcular el valor del tamaño de paso óptimo; en un árbol, un valor por hoja.
  - Actualizar el modelo de ensamble añadiendo el nuevo aprendiz débil multiplicado por el tamaño del paso y por la tasa de aprendizaje $\eta \leq 1$.
  - Calcular los nuevos pseudo-residuos: el gradiente negativo de la pérdida en cada observación (con log-loss, $y - p$).

Gradient boosting a mano: los 14 días

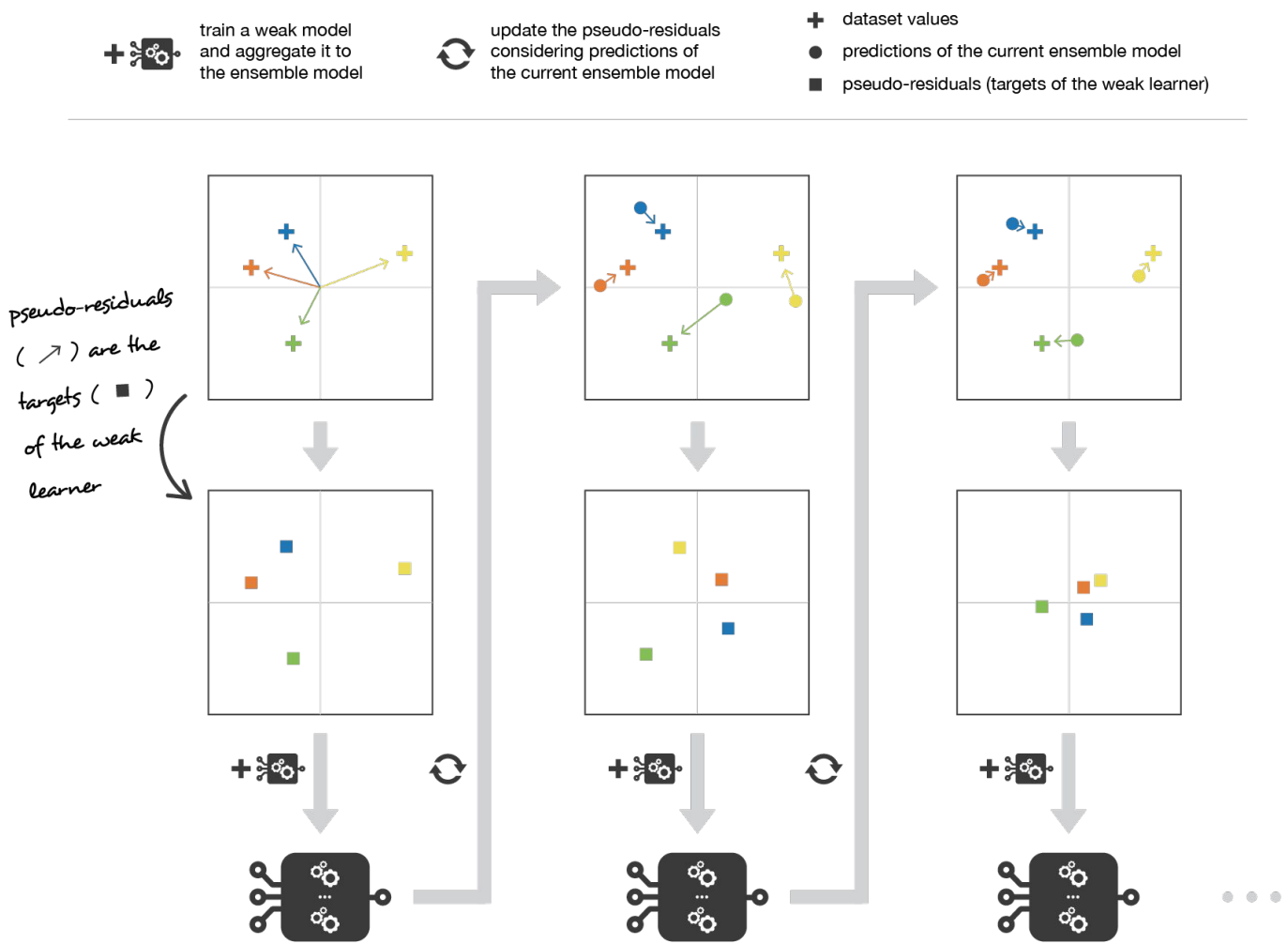
- Log-loss, con 9 días Sí y 5 No. El modelo empieza en el logaritmo de los momios:

$$F_0 = \ln \tfrac{9}{5} = 0.588 \qquad p_0 = \tfrac{9}{14} = 0.643$$

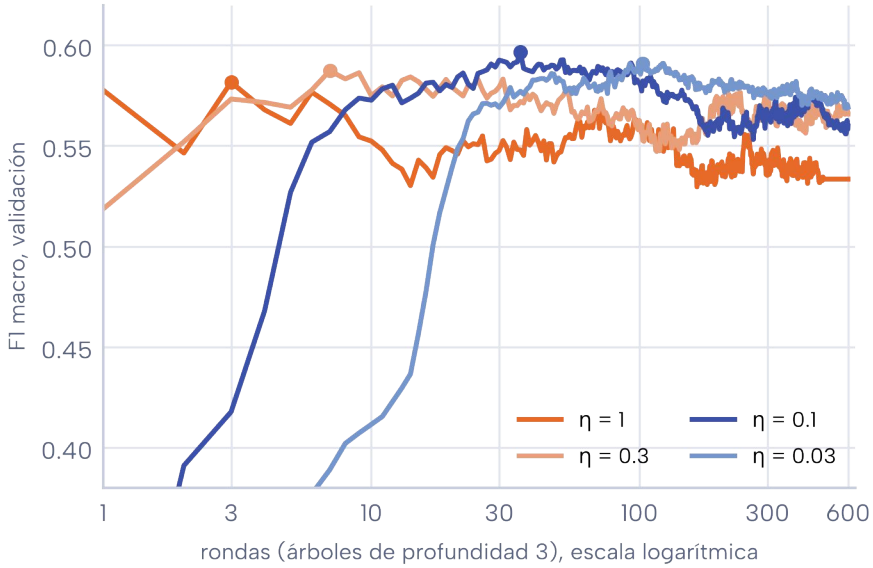
- Pseudo-residuos $y - p_0$: 1 − 0.643 = 0.357 en los 9 días Sí y −0.643 en los 5 No.
- Un tocón de regresión se ajusta a los residuos: el mejor corte separa overcast (4 días, todos Sí) de los otros 10 (5 Sí y 5 No). Cada hoja recibe un paso de Newton:

$$w = \frac{\sum_i (y_i - p_i)}{\sum_i p_i(1 - p_i)} \qquad w_{\text{overcast}} = \frac{4(0.357)}{4(0.230)} = 1.556 \qquad w_{\text{resto}} = \frac{5(0.357) - 5(0.643)}{10(0.230)} = -0.622$$

- Con η = 0.1: F_1 = 0.588 + 0.1(1.556) = 0.743 en overcast, p = 0.678; en los otros 10, 0.588 + 0.1(−0.622) = 0.526, p = 0.628. Cada ronda mueve las probabilidades un poco hacia las proporciones reales de cada hoja.


train a weak model and aggregate it to the ensemble model
update the pseudo-residuals considering predictions of the current ensemble model
dataset values
predictions of the current ensemble model
pseudo-residuals (targets of the weak learner)
pseudo-residuals (↗) are the targets (■) of the weak learner

Tasa de aprendizaje y parada temprana



Los máximos de las curvas se eligen mirando la validación: son optimistas.

- Con η = 1, el mejor F1 de validación (0.582) llega en la ronda 3 y después cae hasta 0.533. Con η = 0.1 llega a 0.597 en la ronda 36; con las 100 rondas por omisión de scikit-learn, 0.581.
- Menor η pide más rondas, pero sobreajusta más despacio. La parada temprana elige las rondas con una validación dentro del entrenamiento.
- Esa validación también debe ir agrupada. La interna de HistGradientBoosting (η = 0.05) aparta 10 % al azar, con gemelos de los dos lados: promete una log-loss de 0.61 y para en 58 rondas, en promedio. Con una partición agrupada: 0.80 y 24 rondas. El F1 casi no cambia (0.572 contra 0.575), pero la interna es optimista.

AdaBoost vs Gradient Boosting

- El *boosting* adaptativo intenta resolver en cada iteración exactamente el problema de optimización **local** (encontrar el mejor aprendiz débil y su coeficiente para añadirlo al modelo fuerte),
- En cambio, el *gradient boosting* utiliza un enfoque de descenso de gradiente y puede adaptarse más fácilmente a un gran número de funciones de pérdida.
- Así, el *gradient boosting* puede considerarse una generalización de *adaboost* para funciones de pérdida diferenciables arbitrarias.

Variantes de Adaboost

- **Discrete AdaBoost**: cada modelo vota −1 o +1 con peso α.
- **Real AdaBoost**: cada modelo da un número real, $\frac{1}{2} \ln(p/(1-p))$, donde p es la probabilidad ponderada de la clase +1 en su hoja; los números se suman.
- **Gentle AdaBoost**: en cada ronda ajusta una regresión por mínimos cuadrados ponderados a $y \in \{-1, +1\}$; sus pasos quedan entre −1 y 1, más conservadores que los de Real AdaBoost.
- **LogitBoost**: minimiza la log-loss con pasos de Newton. Es el antecedente directo del gradient boosting con log-loss.
- **SAMME** y **SAMME.R**: las versiones multiclase. scikit-learn solo conserva SAMME (diapositiva 19).

XGBoost

- XGBoost usa el gradiente *g* y la curvatura *h* de la pérdida en cada ejemplo. Con *G* y *H* sumados en una hoja, su mejor valor y la mejora de un corte son:

$$w^* = -\frac{G}{H+\lambda} \qquad \text{Ganancia} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

- En los 14 días, con p_0 = 9/14: $g_i = p_0 - y_i$ $h_i = p_0(1-p_0) = 0.230$

Corte	**λ = 0: ganancia (hojas)**	**λ = 1: ganancia (hojas)**
Outlook = overcast (4 y 10 días)	**1.556 (1.556 y -0.622)**	0.842 (0.745 y -0.433)
Humidity = normal (7 y 7 días)	1.400 (0.933 y -0.933)	**0.863 (0.575 y -0.575)**

λ castiga a las hojas con poca curvatura: la de overcast tiene H = 4(0.230) = 0.92. Con λ = 0, las hojas son las de la diapositiva 25; con λ = 1, el mejor corte cambia. Y con `min_child_weight` = 1, que también es el valor por omisión, esa hoja ni siquiera se permite.

XGBoost, LightGBM, CatBoost y compañía

Biblioteca	Qué aporta	F1 validación	F1 prueba	Segundos
GradientBoosting	Cualquier pérdida diferenciable; cortes exactos	0.581 ± 0.025	0.610	2.5
XGBoost	Segundo orden, λ y γ, faltantes	0.579 ± 0.032	0.592	0.8
LightGBM	Histogramas, crece hoja por hoja	0.574 ± 0.041	0.598	1.3
CatBoost	Boosting ordenado, categóricas	0.575 ± 0.019	0.639	5.7
HistGradientBoosting	Histogramas al estilo LightGBM, NaN	0.577 ± 0.044	0.630	2.1
AdaBoost, 500 tocones	con SAMME	0.551 ± 0.032	0.550	5.3

- Con valores por omisión, las cinco de gradient boosting empatan en validación (0.574 a 0.581). En prueba van de 0.59 a 0.64: diferencias del tamaño de la variación entre pliegues.
- Con 1279 vinos, todas tardan segundos. Con decenas de miles de filas o más, los histogramas pueden ser órdenes de magnitud más rápidos que los cortes exactos de GradientBoosting.

La unidad 5 en el vino

Modelo	F1 validación	F1 prueba
Sin ensamble (unidad 4)		
Árbol elegido (13)	0.584 ± 0.022	0.551
Naive Bayes (14)	0.579 ± 0.044	0.596
SVM (16)	0.582 ± 0.027	0.576
Árbol completo	0.551 ± 0.038	0.484
Tocón	0.414 ± 0.022	0.389
Bagging y combinación		
Bagging, 500	0.592 ± 0.039	0.618
Random forest, 500	0.592 ± 0.043	0.621
Extra-Trees, 500	0.573 ± 0.053	0.642
Votación suave (los 5)	0.591 ± 0.013	0.572
Stacking (los 5)	0.560 ± 0.019	0.573

Modelo	F1 validación	F1 prueba
Boosting		
AdaBoost, 500 tocones	**0.551 ± 0.032**	**0.550**
GradientBoosting	**0.581 ± 0.025**	**0.610**
HistGradientBoosting	**0.577 ± 0.044**	**0.630**
XGBoost	**0.579 ± 0.032**	**0.592**
LightGBM	**0.574 ± 0.041**	**0.598**
CatBoost	**0.575 ± 0.019**	**0.639**

- Cada ensamble arregla a su modelo base: bagging al árbol completo (0.484 → 0.62 en prueba) y boosting al tocón (0.389 → 0.55 a 0.64).
- En validación nadie se separa: todo queda entre 0.55 y 0.59, con desviaciones de 0.01 a 0.05.
- Con estas 11 variables, los modelos de la unidad 4 parecen estar ya cerca de lo que se puede predecir.

TabArena: ¿qué gana en datos tabulares?

- Un benchmark vivo: 51 conjuntos de datos curados de 1053 candidatos, de 500 a 250,000 filas, 16 modelos y una tabla de posiciones pública que se sigue actualizando.
- Los árboles con gradient boosting siguen entre los mejores: con ajuste de hiperparámetros, gana **CatBoost**.
- Las redes neuronales los alcanzan con más tiempo y ensamble de configuraciones: así, TabM, LightGBM y RealMLP quedan arriba.
- Los modelos fundacionales (TabPFNv2, TabICL) destacan en los conjuntos chicos; TabPFNv2 acepta hasta 10 000 filas.
- El método de validación y el ensamble de configuraciones pesan mucho en el resultado, y un ensamble de todos los modelos supera a cada uno y a AutoGluon.

Es la tesis de la unidad a gran escala: árboles bien ajustados, ensambles de modelos distintos y validación honesta. La Lectura 4 (2022) defendía lo primero; en la P4 lo van a comprobar con la deserción.

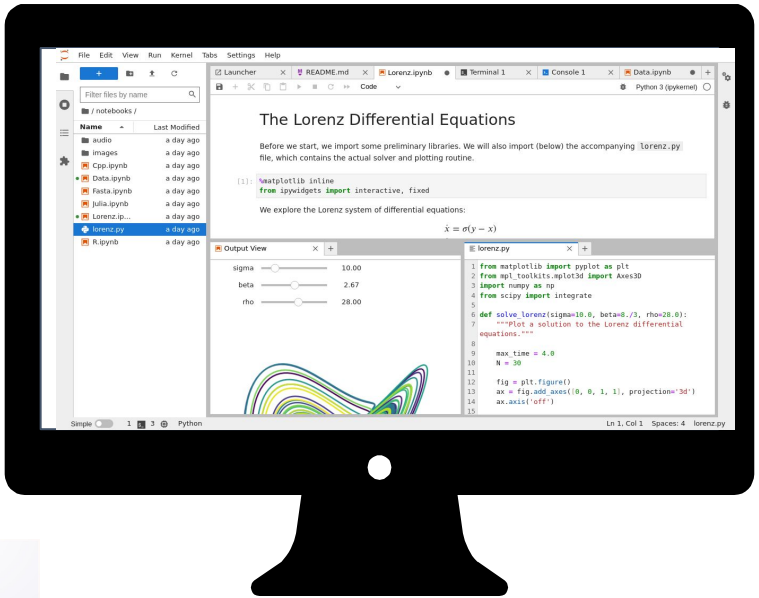
TabPFN: aprender a aprender, con tablas

- TabPFN es un *transformer* preentrenado con millones de tablas sintéticas. Con una tabla nueva no se entrena: recibe juntos el entrenamiento y la prueba, y predice en una pasada (aprendizaje en contexto).
- Es el «meta» de la literatura: aprendió a aprender de tablas.
- En clasificación con hasta 10 000 filas, «en 2.8 s supera a un ensamble de los mejores modelos ajustados durante 4 h».
- TabPFN-2.5 llega a 50 000 filas y 2000 variables y, según la propia empresa, encabeza TabArena. Sus pesos tienen licencia no comercial.

Resumen de Boosting

- Combinar clasificadores débiles para obtener un clasificador muy fuerte:
  - Clasificador débil: ligeramente mejor que el aleatorio en los datos de entrenamiento ($\varepsilon < \frac{1}{2}$; con K clases, $\varepsilon < 1 - 1/K$).
  - Clasificador resultante muy potente: puede llegar a error de entrenamiento cero, pero con demasiadas rondas sobreajusta.
- Boosting vs regresión logística:
  - Optimización única (LR) frente a mejora incremental de la clasificación (B).
- La aplicación más popular de Boosting:
  - Gradient boosting de árboles: XGBoost, LightGBM, CatBoost, HistGradientBoosting.
  - Con tasa de aprendizaje, parada temprana y validación honesta, sigue entre lo mejor en datos tabulares (TabArena).

(Go to live notebook)

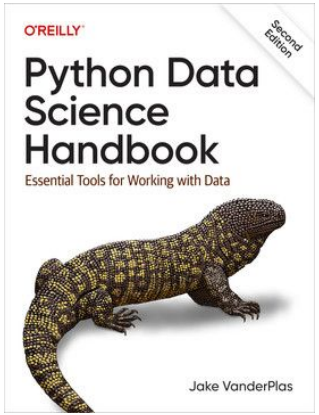


Resumen de Ensambles

	Votación y stacking (5.1)	Bagging (5.2)	Boosting (5.3)
Cómo entrena	Modelos distintos, cada uno por su lado	El mismo modelo en muestras bootstrap, en paralelo	Modelos en secuencia, cada uno sobre los errores de los anteriores
Modelos base	De familias distintas, ya ajustados	Árboles profundos: poco sesgo, mucha varianza	Modelos débiles: tocones o árboles poco profundos
Cómo combina	Voto, o un metamodelo con predicciones fuera del pliegue	Voto o promedio de probabilidades	Suma ponderada: el α de cada modelo o la tasa de aprendizaje
Qué reduce	Los errores que los modelos no comparten	La varianza	El sesgo
Ejemplos	Votación dura y suave, stacking	Bagging, random forest, Extra-Trees	AdaBoost, gradient boosting, XGBoost, LightGBM, CatBoost
Cuidado con	Errores correlacionados; stacking con fugas	El OOB cuando hay datos duplicados	Demasiadas rondas; parada temprana sin grupos

Combinar solo ayuda si los modelos se equivocan distinto. En el vino, cada ensamble arregló a su modelo base sin superar a los mejores de la unidad 4; a gran escala, ganan el boosting bien ajustado y los ensambles de modelos distintos (TabArena).

Extra Libro



GitHub

- ?

Gracias!

¿Alguna pregunta?

hussein@cicese.edu.mx



SCAN ME